



ARISTOTLE
UNIVERSITY
OF THESSALONIKI

Video Game Knowledge Graph

Authors

Georgios Varatis
Evangelia Girousi

Aristotle University of Thessaloniki
M.Sc. Data and Web Science
Semantic Web Technologies

GitHub: <https://github.com/username/video-game-knowledge-graph>

May 2026

Table of Contents

1. Introduction	3
2. Dataset Description.....	3
2.1 Raw Data	3
2.2 Data Volume.....	3
2.3 Preprocessing.....	4
3. Ontology Design.....	4
3.1 Reuse of Existing Ontologies.....	4
3.2 Classes.....	4
3.3 Inferred Classes and OWL Equivalence Axioms	5
3.4 Object Properties.....	5
3.5 Cardinality Constraints	6
4. RML Mapping Rules	6
4.1 IRI Template Convention	6
4.2 Mapping Structure	6
4.3 Programmatic Transformation with RDF4J	7
5. GraphDB Storage and Reasoning Pipeline.....	8
5.1 Repository Loading.....	8
5.2 OWL-Max Reasoning.....	8
5.3 Wikidata Linking.....	8
6. SPARQL Queries and Results.....	8
Q1 — Console Games (OWL Class Inference)	9
Q2 — Indie Games (OWL hasValue Inference).....	9
Q3 — Top Games with Developer and Publisher	9
Q4 — Steam Games with High Metacritic Score	10
Q5 — Creator Directors.....	10
Q6 — Developer Portfolio via Inverse Property.....	11
Q7 — Game Count per Genre.....	11
7. Project Structure and Deliverables.....	11
7.1 Running the Pipeline.....	12
8. Conclusion.....	12

1. Introduction

The report describes the structure and implementation of a full Semantic Web pipeline for the video game domain. The goal of the project is to convert a structured relational dataset into an RDF knowledge graph, store it in a GraphDB triple store with enabled OWL reasoning, and showcase the semantic advantages of this representation with a set of non-trivial SPARQL queries.

The pipeline is implemented in Java with the RDF4J library and consists of four stages including: data pre-processing in Python, RML-based mapping to RDF, programmatic loading in GraphDB with OWL-Max inference, and SPARQL query execution. The schema.org vocabulary is extended with a custom domain-specific OWL 2 vocabulary in the underlying ontology.

2. Dataset Description

The dataset used in this project was sourced from the RAWG Video Games Database API (<https://rawg.io/apidocs>), one of the largest video game databases publicly available. RAWG aggregates data from multiple sources including Metacritic scores, ESRB age ratings, platform availability, and user reviews. The data was retrieved and stored as CSV files covering nine distinct entity types.

2.1 Raw Data

The raw dataset consists of nine CSV files, each corresponding to a core entity in the video game domain. Several of these files contain JSON-embedded nested structures, reflecting the hierarchical nature of the source API response.

File	Description	Key Fields
games.csv	Main game records with nested relationships	id, name, released, metacritic, playtime, ratings, genres, platforms, developers, publishers
genres.csv	Genre categories	id, name, slug, games_count
platforms.csv	Specific platforms (PS5, PC, etc.)	id, name, slug, year_start, year_end
parent_platforms.csv	Platform families (PlayStation, Xbox, etc.)	id, name, slug
developers.csv	Game development studios	id, name, slug, games_count
publishers.csv	Publishing companies	id, name, slug, games_count
creators.csv	Individual creative contributors	id, name, slug
stores.csv	Digital distribution platforms	id, name, slug, domain
tags.csv	User-defined game tags	id, name, slug, language

2.2 Data Volume

After preprocessing, the dataset covers up to 5,000 video games along with all their associated entities. The final RDF output file (output-data.ttl) is approximately 6.8 MB of Turtle-formatted triples. The knowledge graph captures a rich relationship network including which studios

developed each game, which companies published them, which platforms they are available on, which stores sell them, and which creators produced them .

2.3 Preprocessing

The raw CSV files of RAWG embed related entities in columns as JSON arrays, so a preprocessing step was necessary before RML mapping could be used. The file `src/preprocess.py`, a Python script, opens each raw file, extracts and flattens all embedded JSON structures and writes 23 normalized CSV files in the `Dataset/processed/` directory.

These normalized files contain flat entity tables (`genres_flat.csv`, `platforms_flat.csv`, etc.) and explicit join tables representing many-to-many relationships (`game_genres.csv`, `game_platforms.csv`, `developer_games.csv`, etc.). This normalization is mandatory for the RML mapping rules that work on flat, tabular, CSV sources.

3. Ontology Design

The ontology is defined in OWL 2 and using the Turtle serialization format (`videogame-ontology.ttl`). It seeks to model video game domain with appropriate expressiveness to allow meaningful automated inference. The namespace used by the ontology is <http://www.semanticweb.org/videogame-ontology#> and it is an extension of `schema.org` in order to be compatible with existing linked data vocabularies.

3.1 Reuse of Existing Ontologies

Rather than starting from scratch, the ontology reuses and extends a well known ontology. The `schema.org` ontology provides the base classes `schema:VideoGame`, `schema:Organization`, and `schema:Person` and the following properties `schema:name`, `schema:datePublished`, `schema:ratingValue`, and `schema:AggregateRating` are also reused directly. This ensures interoperability with existing linked data on the web and avoids reinventing semantics that are already well-defined.

3.2 Classes

The ontology defines eleven classes covering all major entities in the video game domain:

Class	Extends	Description
VideoGame	<code>schema:VideoGame</code>	A video game title
Developer	<code>schema:Organization</code>	A studio that developed a game
Publisher	<code>schema:Organization</code>	A company that published a game
Creator	<code>schema:Person</code>	An individual creative contributor
CreatorRole	N/A	The role a creator played (e.g. director, writer)
Genre	N/A	A gameplay category (e.g. Action, RPG, Indie)
Platform	N/A	A specific platform (e.g. PlayStation 5, PC)
ParentPlatform	N/A	A platform family (e.g.

		PlayStation, Xbox)
Store	N/A	A digital distribution store (e.g. Steam, Epic)
AggregateRating	schema:AggregateRating	An aggregate user review score
EsrbRating	N/A	An ESRB age classification

Two additional inferred subclasses are defined using OWL equivalentClass axioms. These classes are not asserted explicitly in the data, they are derived entirely by the reasoner at runtime, which is one of the key demonstrations of OWL reasoning in this project.

3.3 Inferred Classes and OWL Equivalence Axioms

ConsoleGame is defined as the set of all VideoGame instances that have at least one hasParentPlatform relationship to a ParentPlatform individual. The equivalentClass axiom uses an owl:intersectionOf with an existential restriction (owl:someValuesFrom):

```
:ConsoleGame owl:equivalentClass [
  owl:intersectionOf (
    :VideoGame
    [ owl:onProperty :hasParentPlatform ;
      owl:someValuesFrom :ParentPlatform ]
  )
] .
```

IndieGame is defined as the set of all VideoGame instances that have the Indie genre linked via hasGenre. The Indie genre is represented as the named individual :genre_51, whose IRI matches the one generated by the RML mapping for genre id 51. The axiom uses an owl:hasValue restriction:

```
:IndieGame owl:equivalentClass [
  owl:intersectionOf (
    :VideoGame
    [ owl:onProperty :hasGenre ;
      owl:hasValue :genre_51 ]
  )
] .
```

When both the ontology and the instance data are loaded into GraphDB with OWL-Max reasoning enabled, every game that satisfies these conditions is automatically assigned the corresponding inferred type, without any explicit typing in the data. SPARQL queries Q1 and Q2 rely entirely on this mechanism and return no results without reasoning.

3.4 Object Properties

The ontology defines thirteen object properties connecting the domain classes. Each property is declared with a domain and range, and six pairs are declared as mutual inverses using owl:inverseOf. The RML mapping only asserts one direction of each inverse pair and the reasoner materializes the other direction automatically.

Property	Domain → Range	Inverse
developedBy	VideoGame → Developer	develops
develops	Developer → VideoGame	developedBy

publishedBy	VideoGame → Publisher	publishes
publishes	Publisher → VideoGame	publishedBy
createdBy	VideoGame → Creator	created
created	Creator → VideoGame	createdBy
hasGenre	VideoGame → Genre	isGenreOf
availableOn	VideoGame → Platform	hosts
onParentPlatform	Platform → ParentPlatform	includesPlatform
hasParentPlatform	VideoGame → ParentPlatform	N/A
soldAt	VideoGame → Store	sells
hasTag	VideoGame → Tag	taggedTo
hasRole	Creator → CreatorRole	N/A

Query Q6 specifically exercises the inverse property mechanism by querying via the `:develops` property, which is never directly asserted in the instance data. The reasoner derives all `:develops` triples from the `:developedBy` assertions in the output file.

3.5 Cardinality Constraints

The VideoGame class carries two minimum cardinality restrictions that enforce structural validity. Every VideoGame instance must have at least one `hasGenre` triple linking it to a Genre, and at least one `availableOn` triple linking it to a Platform. These constraints model the real-world invariant that a video game must belong to at least one genre and be available on at least one platform. Instances violating these constraints are considered logically inconsistent under OWL-Max reasoning.

4. RML Mapping Rules

The transformation from relational CSV data to RDF is defined declaratively in the file `mapping.rml.ttl` using RML. The mapping file contains 19 TriplesMap definitions, each targeting one of the 23 normalized CSV files.

4.1 IRI Template Convention

All entity IRIs follow a consistent template of the form:

```
http://www.semanticweb.org/videogame-ontology#type_id
```

For example, the game with RAWG id 3498 receives the IRI `:game_3498`, and the Indie genre with id 51 receives `:genre_51`. This convention is important because it guarantees that the named individual `:genre_51` defined in the ontology is precisely the same as the IRI produced for the Indie class in the instance data, so that the IndieGame `equivalentClass` restriction is identifiable by the reasoner.

4.2 Mapping Structure

Each TriplesMap in the mapping file specifies a logical source (the CSV file), a subject map defining the IRI template and RDF type for each row, and a set of predicate-object maps connecting columns to ontology properties. The following table summarizes the key mappings:

TriplesMap	Source CSV	RDF Class	Key Predicates
GameMap	games_flat.csv	:VideoGame	schema:name, schema:datePublished, :metacriticScore, :playtime, :ratingsCount
GenreMap	genres_flat.csv	:Genre	schema:name, :slug, :gamesCount
PlatformMap	platforms_flat.csv	:Platform	schema:name, :slug, :yearStart, :yearEnd
ParentPlatformMap	parent_platforms_flat.csv	:ParentPlatform	schema:name, :slug
StoreMap	stores_flat.csv	:Store	schema:name, :slug, :storeDomain
TagMap	tags_flat.csv	:Tag	schema:name, :slug, :language
DeveloperMap	developers_flat.csv	:Developer	schema:name, :slug, :gamesCount
PublisherMap	publishers_flat.csv	:Publisher	schema:name, :slug, :gamesCount
CreatorMap	creators_flat.csv	:Creator	schema:name, :slug
CreatorRoleMap	creator_roles_flat.csv	:CreatorRole	schema:name, :slug
AggregateRatingMap	game_ratings.csv	:AggregateRating	schema:ratingValue, linked via :hasRating
EsrbRatingMap	game_esrb.csv	:EsrbRating	schema:name, :slug, linked via :hasEsrbRating
GameGenreMap	game_genres.csv	N/A	:hasGenre (VideoGame → Genre)
GamePlatformMap	game_platforms.csv	N/A	:availableOn (VideoGame → Platform)
GameParentPlatMap	game_parent_platforms.csv	N/A	:hasParentPlatform (VideoGame → ParentPlatform)
GameStoreMap	game_stores.csv	N/A	:soldAt (VideoGame → Store)
GameTagMap	game_tags.csv	N/A	:hasTag (VideoGame → Tag)
DeveloperGameMap	developer_games.csv	N/A	:develops (Developer → VideoGame)
CreatorGameMap	creator_games.csv	N/A	:created (Creator → VideoGame)

4.3 Programmatic Transformation with RDF4J

A completely declarative RML processor was considered, but eventually replaced by a script in Java (RMLProcessor.java). This was done for several practical reasons.

First, the raw RAWG data required non-trivial preprocessing that falls outside what standard RML processors handle cleanly, such as parsing floating-point strings like "2020.0" and converting them to integer year values. Second, the dataset needed selective filtering: only developers, publishers, and creators related to the selected game subset should be included in

the output, which requires cross-file join logic not expressible in plain RML. Third, streaming output was required to handle the 6.8 MB output without building a full in-memory RDF model, which would risk memory exhaustion.

The script relies on Apache Commons CSV for parsing, and the RDF4J's Rio streaming writer for outputting the triples directly to the output file as each CSV row is processed. The `mapping.rml.ttl` file is the standard specification of the mapping rules.

5. GraphDB Storage and Reasoning Pipeline

The Java pipeline (`Main.java`) orchestrates three sequential steps: RML transformation, GraphDB loading, and SPARQL query execution. GraphDB is configured with a repository named `MiniProject` using the OWL-Max reasoning profile.

5.1 Repository Loading

The `GraphDBConnector` class connects to GraphDB via RDF4J's `HTTPRepository` interface. Before each run, the repository is cleared to ensure a clean state. Three files are then loaded in sequence: the ontology (`videogame-ontology.ttl`), the instance data (`output-data.ttl`), and the Wikidata external links (`sameAs_wikidata.ttl`).

Because the instance file is approximately 6.8 MB, uploading it as a single HTTP request would exceed typical request body size limits. The connector therefore parses the file into an in-memory RDF4J Model and uploads it in batches of 5,000 statements per HTTP request, ensuring reliable transfer regardless of the repository configuration.

5.2 OWL-Max Reasoning

GraphDB's OWL-Max reasoning profile is selected because it supports the full set of OWL features used in the ontology: `equivalentClass`, `owl:someValuesFrom`, `owl:hasValue`, `owl:inverseOf`, and `owl:FunctionalProperty`. Once the ontology and instance data are loaded, GraphDB performs forward-chaining inference and materializes all inferred triples into the repository. This means that queries run against the repository see both explicit and inferred triples, enabling the `ConsoleGame` and `IndieGame` classification queries to return results.

5.3 Wikidata Linking

The Python script `src/link_to_wikidata.py` extends the ontology by querying the Wikidata SPARQL endpoint to find matching video game entities. For each match, an `owl:sameAs` triple is produced between the local game IRI and the corresponding Wikidata URI. The generated `sameAs_wikidata.ttl` file is loaded together with the main instance data and thus enables federated queries on the local knowledge graph and Wikidata.

6. SPARQL Queries and Results

Seven SPARQL queries are defined, each demonstrating a distinct semantic capability that would not be achievable with simple relational queries. The queries are stored as separate `.sparql` files

in the Java resources directory and are executed programmatically by the SPARQLRunner class, which prints the first 25 result rows per query.

Q1 — Console Games (OWL Class Inference)

This query retrieves all games classified as :ConsoleGame together with their parent platform names. No game is explicitly typed as ConsoleGame in the data. The results are produced entirely by the OWL-Max reasoner applying the equivalentClass axiom. Without reasoning enabled, this query returns zero results.

```
PREFIX : <http://www.semanticweb.org/videogame-ontology#>
PREFIX schema: <https://schema.org/>

SELECT DISTINCT ?gameName ?parentPlatformName
WHERE {
    ?game a :ConsoleGame ;
        schema:name ?gameName ;
        :hasParentPlatform ?pp .
    ?pp schema:name ?parentPlatformName .
}
ORDER BY ?gameName ?parentPlatformName
LIMIT 25
```

Q2 — Indie Games (OWL hasValue Inference)

This query retrieves games classified as :IndieGame by the hasValue restriction on :genre_51. The optional Metacritic score is included to rank results. Like Q1, the IndieGame type is never asserted in the data and depends entirely on the reasoner.

```
PREFIX : <http://www.semanticweb.org/videogame-ontology#>
PREFIX schema: <https://schema.org/>

SELECT DISTINCT ?gameName ?score
WHERE {
    ?game a :IndieGame ;
        schema:name ?gameName .
    OPTIONAL { ?game :metacriticScore ?score }
}
ORDER BY DESC(?score)
LIMIT 25
```

Q3 — Top Games with Developer and Publisher

This query demonstrates a multi-class join: for each highly-rated game it retrieves the developer and publisher by traversing the :develops and :publishes inverse properties. The result set shows game name, developer name, publisher name, and Metacritic score, ordered by score descending.

```
PREFIX : <http://www.semanticweb.org/videogame-ontology#>
PREFIX schema: <https://schema.org/>

SELECT DISTINCT ?gameName ?devName ?pubName ?score
```

```

WHERE {
    ?game a :VideoGame ;
          schema:name ?gameName ;
          :metacriticScore ?score .
    ?dev :develops ?game ; schema:name ?devName .
    ?pub :publishes ?game ; schema:name ?pubName .
}
ORDER BY DESC(?score)
LIMIT 25

```

Q4 — Steam Games with High Metacritic Score

This query filters VideoGame instances available on Steam with a Metacritic score above 80. It demonstrates combined object property traversal and numeric datatype filtering.

```

PREFIX : <http://www.semanticweb.org/videogame-ontology#>
PREFIX schema: <https://schema.org/>

SELECT DISTINCT ?gameName ?score
WHERE {
    ?game a :VideoGame ;
          schema:name ?gameName ;
          :metacriticScore ?score ;
          :soldAt ?store .
    ?store schema:name "Steam" .
    FILTER (?score > 80)
}
ORDER BY DESC(?score)
LIMIT 25

```

Q5 — Creator Directors

This query traverses two object properties simultaneously: :hasRole links a Creator to a CreatorRole, and :created links the same Creator to a VideoGame. A FILTER on the role name isolates directors specifically.

```

PREFIX : <http://www.semanticweb.org/videogame-ontology#>
PREFIX schema: <https://schema.org/>

SELECT DISTINCT ?creatorName ?roleName ?gameName
WHERE {
    ?creator a :Creator ;
             schema:name ?creatorName ;
             :hasRole ?role ;
             :created ?game .
    ?role schema:name ?roleName .
    ?game schema:name ?gameName .
    FILTER (LCASE(?roleName) = "director")
}
ORDER BY ?creatorName ?gameName
LIMIT 25

```

Q6 — Developer Portfolio via Inverse Property

The `:develops` property is never asserted in the instance data, only `:developedBy` is. This query uses `:develops` directly, which only works because the OWL-Max reasoner has materialized the inverse triples from the `owl:inverseOf` declaration. Without reasoning, this query returns no results.

```
PREFIX : <http://www.semanticweb.org/videogame-ontology#>
PREFIX schema: <https://schema.org/>

SELECT DISTINCT ?devName ?gameName ?score
WHERE {
    ?dev a :Developer ;
        schema:name ?devName ;
        :develops ?game .
    ?game schema:name ?gameName .
    OPTIONAL { ?game :metacriticScore ?score }
}
ORDER BY ?devName DESC(?score)
LIMIT 25
```

Q7 — Game Count per Genre

This query uses SPARQL aggregation (`COUNT` and `GROUP BY`) over the `:hasGenre` property to compute how many games belong to each genre, ordered by popularity. It demonstrates that the knowledge graph supports analytical queries over the full dataset, going beyond simple entity lookup.

```
PREFIX : <http://www.semanticweb.org/videogame-ontology#>
PREFIX schema: <https://schema.org/>

SELECT ?genreName (COUNT(DISTINCT ?game) AS ?gameCount)
WHERE {
    ?genre a :Genre ; schema:name ?genreName .
    ?game :hasGenre ?genre .
}
GROUP BY ?genreName
ORDER BY DESC(?gameCount)
```

7. Project Structure and Deliverables

The project is organized as follows:

```
Ontology Project/
├── Dataset/
│   ├── raw_data/           (9 raw RAWG CSV files)
│   └── processed/         (23 normalized CSV files)
├── MiniProject/           (Maven Java project)
│   ├── pom.xml
│   └── src/main/
│       └── java/ontology/
```

```

├── Main.java
├── RMLProcessor.java
├── GraphDBConnector.java
├── SPARQLRunner.java
├── resources/queries/
│   ├── q1_console_games.sparql
│   ├── q2_indie_games.sparql
│   ├── q3_top_games_dev_pub.sparql
│   ├── q4_steam_metacritic.sparql
│   ├── q5_creator_directors.sparql
│   ├── q6_developer_develops.sparql
│   └── q7_games_per_genre.sparql
├── src/
│   ├── preprocess.py
│   └── link_to_wikidata.py
├── videogame-ontology.ttl
├── mapping.rml.ttl
├── output-data.ttl
└── sameAs_wikidata.ttl

```

The Java project is built with Maven and requires Java 11 or later. The main dependencies are RDF4J 4.3.10 (for the RDF model, Rio streaming writer, and HTTP repository connector) and Apache Commons CSV 1.10.0 (for CSV parsing). GraphDB must be running locally on port 7200 with a repository named MiniProject configured with the OWL-Max reasoning profile.

7.1 Running the Pipeline

To run the full pipeline:

- Ensure GraphDB is running at <http://localhost:7200> with the MiniProject repository created
- Run: `mvn package` from the MiniProject/ directory
- Run: `java -jar target/MiniProject-1.0-SNAPSHOT.jar`
- Optionally override the project root: `java -DPROJECT_DIR="path/to/project" -jar ...`

8. Conclusion

In this project, we show a full end-to-end Semantic Web pipeline on a real video game dataset. The pipeline transforms raw relational CSV data from the RAWG API into a highly connected RDF knowledge graph based on a custom OWL 2 ontology extending schema.org.

First, OWL class inference implicitly classifies games as ConsoleGame or IndieGame according to their attributes without explicit type assertions in the data. Second, inverse property materialization enables queries in both directions of a relationship, even if only one direction is present in the source data. Third, the Wikidata owl:sameAs links connect the local knowledge graph to the broader linked data network, allowing identity resolution and federated querying across knowledge sources.

Together, these capabilities demonstrate that the semantic layer provides actual value that is not possible with the original CSV data and traditional SQL queries.